

SHOOLIN AI

Smart Entry. Smart Compliance. Smart School.

JABARINATHAN V – 24623157

SIVANESAN M -24623157

THARUN S-24623142

VELS INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**SHOOLIN AI**", is the Bonafide work of **JABARINATHAN V - 24623157**, **SIVANESAN M-24623159** and **THARUN S-24623142** carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya M.E., Ph.D
Associate professor & Head Professor
Department of CSE (AI)
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVISOR

Dr. S.J.Kavitha
Assistant professor
Department of CSE (AI)
School of Engineering
VISTAS
Chennai

Submitted for the Project Work and Viva - Voce examination held on……

at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our **FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH**, and it is with great pleasure that we take this opportunity to thank him. We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya**, for the continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. We extend our warmest thanks to our **INTERNAL GUIDE,** Dr. S.J.Kavitha, who has patiently guided us and provided invaluable advice and help. We would like to express our greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of the Computer Centre, who have helped and supported us throughout. Last but not the least, we wish to thank our parents and friends for their individual support and interest, who have inspired us and encouraged us to go in our own way; without them we would be unable to complete this project.

JABARINATHAN V – 24623157

SIVANESAN M -24623157

THARUN S-24623142

CHAPTER NO.	TABLE OF CONTENT	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	ii
	ABBREVIATIONS	iii
1	**INTRODUCTION**	1
	1.1 Overview of the project	1
	1.2 Scope and objective	3
2	**LITERATURE REVIEW**	5
	2.1 Literature Overview	5
	2.2 Literature Survey	5
3	**SYSTEM DESIGN**	7
	3.1 Existing System	7
	3.2 Proposed System	7
	3.3 System Architecture	8
	3.4 Hardware Requirements	11
	3.5 Software Requirements	11
4	**IMPLEMENTATION**	13
	4.1 Methodology	13
	4.2 Implementation	15
	4.2.1 Image Input & Preprocessing Module	15
	4.2.2 Object Detection Module	15
	4.2.3 Compliance Rule Engine	15
	4.2.4 Entry Logging & Review Module	16
	4.2.5 Dashboard & Notification Module	16
5	**CONCLUSION AND FUTURE WORK**	17
	5.1 Conclusion	17
	5.2 Future Work	18
	5.3 Source Code	19
	5.4 Results	21
	REFERENCES	23

ABSTRACT

Schools often check student identity cards, uniforms, belts and shoes manually at the entrance. This process is time-consuming, inconsistent and difficult to record accurately when many students arrive together. **Shoolin AI** proposes an AI-assisted school-entry monitoring system that examines a camera or uploaded image, uses a YOLO object-detection model with OpenCV to look for four required items (ID card, uniform, belt and shoes), applies a transparent, school-configurable rule engine, and records the date, time and result of every check in a searchable entry history that staff can review on a dashboard. Uncertain results are sent to staff for manual review, so the system supports staff decisions rather than replacing them. **Current status:** the system architecture, compliance logic, entry-logging design and a staff-checklist demonstration prototype have been prepared; training and evaluation of the custom detection model on a labelled school dataset, and parent notifications through an approved messaging service, are pending and are described as future work.

Keywords Shoolin AI, school entry monitoring, computer vision, object detection, YOLO, OpenCV, uniform compliance, staff dashboard, manual review.

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO.
Figure 1	Proposed system architecture of Shoolin AI	9
Figure 2	Compliance decision logic	9
Figure 3	Staff dashboard user interface	21
Figure 4	Item-wise detection result screen	22
Figure 5	Entry history and staff review screen	22

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
YOLO	You Only Look Once
OpenCV	Open Source Computer Vision Library
ID	Identity
mAP	Mean Average Precision
IoU	Intersection over Union
GUI	Graphical User Interface
API	Application Programming Interface
CSV	Comma-Separated Values
SQL	Structured Query Language
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
GPU	Graphics Processing Unit
RAM	Random Access Memory
SMS	Short Message Service
DPDP	Digital Personal Data Protection (Act, 2023)

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Introduction

The entrance of a school is where discipline begins each day. Staff and prefects check whether students are wearing the ID card, the approved uniform, the school belt and the correct shoes, and they note the arrival time. When many students arrive within a few minutes, this manual checking becomes slow and inconsistent, and paper records are difficult to search later.

Shoolin AI (tagline: "Smart Entry. Smart Compliance. Smart School.") is a proposed AI-powered school-entry monitoring system. It uses computer vision to examine an image of the student, identifies whether the four required items appear to be present, records the entry time, and shows an understandable status to school staff. The system is meant to **support staff decisions** and not to make unsupported assumptions about a student.

Problem Statement

Schools often check student identity cards, uniforms, belts, shoes and arrival times manually at the entrance. This has several limitations:

- **Time-Consuming:** Checking each student by eye takes time when many students arrive at once.
- **Inconsistent:** Different staff members may apply the rules differently.
- **Poor Records:** Paper records make it hard to review attendance and to identify repeated compliance issues.

The core problem is the lack of a consistent, digital and assistive entry-checking process with a reviewable history.

Objectives of the Project

The main objectives of this project are:

- To develop a smart school-entry monitoring prototype.
- To detect the student and the relevant compliance items (ID card, uniform, belt and shoes) from images or camera input.
- To display an understandable compliance result for each item and an overall status.
- To record the date and time of every entry check and maintain a searchable history.
- To provide a staff dashboard for reviewing results and correcting mistakes.
- To support future parent notifications through an approved messaging service.
- To protect student images and personal information through appropriate access controls.

Scope of the Project

The system is designed to **assist** school staff, not to replace them. Its scope highlights include:

- **Image Capture:** A camera, webcam or authorised uploaded image.
- **Detection:** Four classes – id_card, uniform, belt and shoes (plus an optional person class).
- **Rule-Based Status:** A configurable rule engine produces the entry status.
- **Records:** Date, time, item-wise result and review status are stored.
- **Applicability:** School entry gates and designated checking areas.

System Overview

The system integrates the following major components:

1. **Image Input and Preprocessing:** Checks whether the image is usable and prepares it for the model.
2. **YOLO Object Detection:** Detects the student and the required items and gives a confidence score for each detection.
3. **Compliance Rule Engine:** Combines detections with school rules to decide which items appear present, missing or uncertain.
4. **Entry Log and Dashboard:** Stores the result with a timestamp and lets staff review and correct it.

Advantages of the System

The intended advantages are listed below. They are expected benefits; real improvements will be stated only after they are measured.

- **Faster Initial Checks:** Reduces repetitive manual work.
- **Consistent Rules:** The same configured rules are applied to every student.
- **Digital History:** Easier review of arrival times and recurring issues.
- **Staff in Control:** Uncertain results go to manual review.

Applications of the Project

- **School Entrance Gates:** Daily entry checking.
- **Attendance Records:** A digital arrival log.
- **Discipline Monitoring:** Review of recurring uniform-compliance issues.
- **Future School Automation:** A base for parent alerts and reports.

Expected Outcomes

- A working architecture and rule engine for entry compliance.
- A staff dashboard showing item-wise results, status and history.

- A trained custom YOLO model **(pending – to be trained and evaluated)**.
- A foundation for further school-automation research.

Future Enhancements

- Live webcam integration and real-time multi-student detection.
- Staff login, student database integration and parent notifications.
- Reports and analytics, and school-specific uniform configuration.

1.2 SCOPE AND OBJECTIVE

Purpose of the Study

The purpose of this project is to design and prototype an assistive, privacy-aware entry-checking system for schools. This is pursued by:

1. Using computer vision to look for the four required items in an image.
2. Applying transparent rules and a manual-review path instead of automatic denial.
3. Keeping a digital, reviewable record of every check.

General Objective

The general objective is to build a reliable, understandable and privacy-conscious system that helps staff perform entry checks faster and more consistently, while keeping the final decision with a human.

Specific Objectives

- To collect authorised images and label them with bounding boxes for id_card, uniform, belt and shoes.
- To train a YOLO model on the labelled dataset and evaluate it on an unseen test set.
- To design a configurable rule engine with detected, not detected and uncertain states.
- To store each check in a database or log with a timestamp and review status.
- To build a dashboard for results, history and settings.
- To test the system with real cases and report only measured results.

Scope of the Project

A. Functional Scope

- **Student Entry Check:** Student identifier, image upload or capture, item-wise results, confidence scores, timestamp and overall status.
- **Manual Review:** Staff can correct mistakes and confirm the final decision.
- **Entry History:** Searchable list of checks.
- **Settings:** School rules, confidence threshold, data retention and authorised users.

B. Technical Scope

- **Language and Libraries:** Python, OpenCV and a YOLO model (Ultralytics).
- **Interface:** HTML, CSS and JavaScript, or Streamlit for a simpler prototype.
- **Storage:** SQLite for a local demonstration; a CSV log for the simplest prototype.
- **Version Control:** Git and GitHub.

C. Out of Scope in the Current Prototype

- Verified automatic identification of a student's identity.

- Sending real SMS or WhatsApp messages (future scope).
- Production-level security and multi-student real-time detection.

Importance of the Project

The project addresses a real, everyday school-administration task with a clear computer-vision application. It also shows how AI can be used responsibly: as an assistant with confidence scores, a manual-review path and strong privacy controls.

The Four Detection Classes

Class	Purpose
id_card	Detect the required student identity card
uniform	Detect the approved school uniform
belt	Detect the required school belt
shoes	Detect the required school shoes

A separate person class may also be useful. These class names do not mean the model can already detect the items reliably; training, validation and real-world testing are required.

Hardware and Software Requirements

- **Hardware:** Windows 11 computer (Intel Core i5 or higher, 8 GB RAM or more), webcam or camera; a GPU is recommended for training.
- **Software:** Python, OpenCV, Ultralytics YOLO, SQLite, a browser, VS Code and Git.

Challenges and Limitations

- Custom training is required for reliable detection of school items.
- Small items such as ID cards, belts and shoes are hard to detect.
- Poor lighting, camera angles and occlusion reduce accuracy.
- A missing detection is not proof that an item is absent.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

Computer vision has moved from hand-designed features to deep neural networks that detect many kinds of objects in a single image. Object detection is the task of finding objects and drawing a bounding box with a class label and a confidence score around each. It is used in retail, traffic, safety-equipment checking and many other fields, and it is the core technique needed by Shoolin AI.

This review studies how object-detection methods developed, which tools are suitable for a student prototype, and what issues arise when such tools are used in a school. It serves the following purposes:

- To understand the evolution of object-detection algorithms and choose a suitable model.
- To understand the metrics used to evaluate detectors.
- To identify privacy considerations when images of children are processed.
- To provide the justification for the proposed system.

2.2 LITERATURE SURVEY

Early Object Detection

Viola and Jones [4] introduced a boosted cascade of simple features that made real-time face detection practical on ordinary computers. Such methods work well for faces but are not designed for objects with varied appearance, such as belts, shoes or uniforms.

Two-Stage and One-Stage Deep Detectors

Region-based detectors such as Faster R-CNN [3] first propose regions and then classify them, giving high accuracy but at greater computational cost. Redmon et al. [1] proposed YOLO, which treats detection as a single regression problem and predicts bounding boxes and class probabilities directly from the full image in one pass. This makes YOLO fast enough for live camera use. Newer versions are maintained in the Ultralytics framework [2], which provides pretrained weights, training and evaluation tools.

Datasets and Evaluation Metrics

Pretrained detectors are commonly trained on large datasets such as Microsoft COCO [5], which includes the class “person” but not school-specific items such as ID cards or belts. This is why Shoolin AI needs a custom labelled dataset. Detectors are evaluated using precision, recall and mean Average Precision (mAP), following the Pascal VOC challenge methodology [6].

OpenCV for Image Handling

The OpenCV library [7] provides functions for reading camera frames, resizing, colour conversion and drawing results, and is the standard tool for connecting a detector to a camera.

Privacy and Ethics when Processing Children's Images

Images of students are personal data. India's Digital Personal Data Protection Act, 2023 [8] places specific obligations, including parental consent, on the processing of children's data. Any real deployment must therefore obtain school and parental approval, limit access and retention, and avoid publishing identifiable images. Project authors should confirm the exact legal requirements with their institution.

Research Gaps Identified

- **School-Specific Items:** General detectors do not cover ID cards, belts and uniforms of a specific school.
- **Assistive Design:** Many demonstrations present AI output as final; a transparent rule engine with manual review is less common.
- **Privacy by Design:** Student-image handling is often ignored in prototypes.

This project addresses these gaps by combining a custom YOLO model with a configurable rule engine, a manual-review path and privacy-conscious data handling.

Summary of Literature Findings

The literature shows that one-stage detectors such as YOLO are fast, widely available and suitable for a camera-based prototype, but they must be trained on suitable labelled data and evaluated with proper metrics. Using them with children's images requires careful privacy practice. Shoolin AI follows these findings.

CHAPTER 3. SYSTEM DESIGN

3.1 EXISTING SYSTEM

Overview

In most schools the entrance check is manual. Staff or prefects look at each student and may record defaulters on paper or in a register.

Limitations of Traditional Methods

- **Slow:** Checking many students at once causes queues.
- **Inconsistent:** Judgement varies between staff members.
- **Limited Records:** Paper records are hard to search and summarise.
- **No Pattern Detection:** Repeated issues are difficult to identify.

Common Issues in Existing Systems

- Human error and fatigue during busy periods.
- No digital timestamp for each check.
- Difficult communication of issues to parents.

3.2 PROPOSED SYSTEM

Overview

The proposed Shoolin AI system captures an image of the student, detects the required items with a YOLO model, evaluates them with configured rules, and shows a status with confidence scores. Every check is logged with a timestamp, and staff can review and correct the result.

Objectives

- To help staff perform entry checks consistently.
- To keep a digital history of entry checks.
- To flag uncertain cases for human review.

Features of the Proposed System

- **Four-Item Detection:** id_card, uniform, belt and shoes.
- **Item-Wise Result:** Each item is shown as detected, not detected or uncertain.
- **Overall Status:** ALLOWED / COMPLIANT, DEFAULTER – CHECK REQUIRED, or MANUAL REVIEW REQUIRED.
- **Timestamped Logging:** Student identifier, time, results and review status.
- **Staff Dashboard:** Summary, entry check, history and settings.
- **Optional Notification (Future):** Parent message through an approved provider.

Advantages

- Faster initial checks and consistent recordkeeping.
- Transparent decisions with an explanation.
- A digital basis for reports and later improvements.

Applications

- School entrance gates and designated checking areas.
- Daily arrival records and discipline reviews.

3.3 SYSTEM ARCHITECTURE

Architectural Overview

The system follows a modular pipeline so that each stage can be built, tested and improved independently. The image flows from the camera or upload through preprocessing and the YOLO model to the rule engine; the result is then stored and shown to staff.

Architectural Flow

Step 1: Image Capture

A camera, webcam or authorised uploaded image captures the student.

Step 2: Preprocessing

The software checks that the image is usable (not too dark or blurry) and prepares it for the model.

Step 3: AI Detection

A trained YOLO model identifies the student and attempts to detect id_card, uniform, belt and shoes.

Step 4: Compliance Evaluation

Detections and confidence scores are combined with configured rules to decide which items appear present, missing or uncertain.

Step 5: Entry Result and Logging

The dashboard shows the status and the application records the student identifier, timestamp and result.

Step 6: Staff Review and Notification

Authorised staff correct mistakes and confirm the final decision. In a future version, an approved notification can be sent to the parent or guardian.

System Architecture Diagram

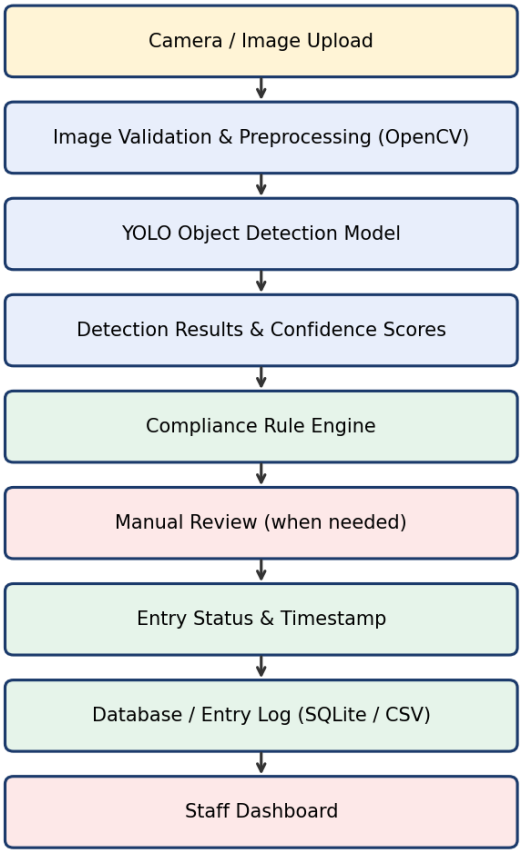


Figure 1 - Proposed system architecture

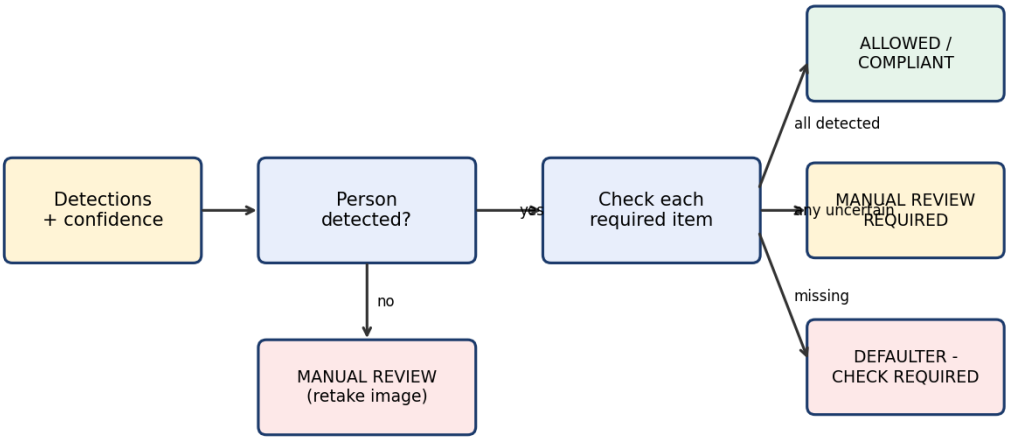


Figure 2 - Compliance decision logic

Compliance Decision Logic

- If all four required items are confidently detected, the result can be marked compliant.

- If one or more items are confidently identified as missing, the result is marked for a compliance check.
- If detection is uncertain or the image is unusable, manual review is requested.

A student is never denied entry automatically only because the model failed to detect an item; the system assists staff, explains the result and allows correction.

3.4 HARDWARE REQUIREMENTS

Minimum Hardware Configuration (Prototype)

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5
RAM	8 GB
Storage	256 GB SSD
Camera	720p Webcam

Recommended Hardware Configuration (Training / Deployment)

Component	Specification
Processor	Intel Core i7 / AMD Ryzen 7 or higher
RAM	16 GB or more
Storage	512 GB SSD
GPU	NVIDIA GPU (recommended for faster YOLO training)
Camera	1080p USB / IP Camera

Hardware Justification: Inference with a small YOLO model can run on a CPU for single-image checks, but training the custom model is much faster on a GPU.

3.5 SOFTWARE REQUIREMENTS

Software Tools

Software	Description
Operating System	Windows 11
Programming Language	Python
Core Libraries	
OpenCV	Image handling and camera integration
Ultralytics YOLO	Object detection training and inference
Application	
HTML, CSS, JavaScript / Streamlit	Staff interface
SQLite / CSV	Entry records
Tools	
VS Code, PowerShell	Development environment
Git and GitHub	Version control

Software Modules

- **data.yaml / train.py:** Dataset configuration and YOLO training.
- **detector.py:** Runs the model on an image and returns detections.
- **compliance.py:** The rule engine that produces item-wise and overall status.
- **entry_log.py:** Saves each check to the database.
- **Interface files:** index.html, style.css, script.js (or app.py in the Streamlit prototype).

Database Design

A simple starting schema is proposed. Only the information required for the project's purpose should be stored, and no real phone numbers or sensitive data should be placed in a public repository.

Table	Main Fields
Students	student_id, student_name, class_name
Entry checks	check_id, student_id, check_timestamp, id_card_status, uniform_status, belt_status, shoes_status, overall_status, review_status
Detection records	detection_id, check_id, class_name, confidence_score, detection_status
Notification records (optional)	notification_id, student_id, check_id, sent_timestamp, delivery_status

CHAPTER 4. IMPLEMENTATION

4.1 METHODOLOGY

The methodology defines the systematic approach for developing Shoolin AI. It covers data collection, labelling, model training, evaluation, rule design and integration. Steps that have not yet been carried out are clearly marked as **pending** so that this report does not claim results that do not exist.

Research Approach

The project is applied and experimental. It combines a computer-vision model with a rule-based decision layer and a staff review step.

Dataset Preparation

1. Collect authorised sample images of the school uniform, ID card, belt and shoes.
2. Remove unnecessary personal information and duplicate images.
3. Label images with bounding boxes for id_card, uniform, belt and shoes.
4. Divide the dataset into training, validation and test sets.

The intended dataset folder structure is Shoolin_AI/AI/dataset/ with images/train, images/val, labels/train and labels/val; a separate test split is recommended for a stronger evaluation. Each image must have a matching label file containing the class index and normalised bounding-box coordinates. Only images collected with appropriate authorisation are used, and identifiable student images are not published.

Status: Dataset collection is in progress; the number of images per class and train-validation-test split will be updated after collection is completed.

Model Training and Evaluation

- **Model:** A YOLO model (starting from yolo11n.pt pretrained weights) is trained on the labelled dataset using train.py and data.yaml.
- **Observation So Far:** The pretrained model was observed detecting a **person**. It must not be assumed to detect the custom classes id_card, uniform, belt or shoes.
- **Evaluation:** The trained weights are evaluated on unseen test images using precision, recall, F1 score, mAP and inference time, reported for each class separately.
- **Status: Training and evaluation are pending.** No accuracy values are claimed in this report.

Compliance Rule Design

Two confidence limits are configured: detections at or above the upper limit are treated as “detected”, those between the two limits as “uncertain”, and those below the lower limit as “not detected”. The overall status is derived from the four item results as described in Section 3.3.

Because a missing detection is not always proof that an item is absent, uncertain cases go to staff for manual review.

Testing Plan

The system will be tested with the cases in Section 5.4. The actual result, pass/fail status and evidence are recorded for each case; no results are invented.

Ethical and Privacy Considerations

- **Authorisation and Consent:** Use only images collected with school and parental approval.
- **Access Control:** Only authorised staff can see student details; the dashboard is not publicly accessible.
- **Minimal Data and Retention:** Store only what is needed and set a retention period.
- **Human in the Loop:** The system does not make final entry decisions.

4.2 IMPLEMENTATION

The implementation consists of independent modules. The table below separates what has been prepared from what is still pending.

Component	Status	Note
Architecture and workflow design	Prepared	Chapter 3
Rule engine (compliance.py)	Written and tested with sample inputs	Logic only
Staff checklist demo (Streamlit)	Prepared	Confirmed
Custom YOLO training	Pending	Needs labelled data
Model evaluation	Pending	Needs test set
Parent notification	Future scope	Needs provider, consent

The checklist demonstration lets staff tick the four items manually and shows a status. It **does not analyse images automatically** and must not be presented as AI detection.

4.2.1 Image Input & Preprocessing Module

Technology Used: OpenCV

- **Capture:** cv2.VideoCapture(0) for a webcam, or an uploaded image.
- **Validation:** Reject images that are unreadable, too dark or too blurry and request another image.
- **Preparation:** Resize and normalise the image for the model.

Output: A clean image ready for detection.

4.2.2 Object Detection Module

Technology Used: Ultralytics YOLO

- **Model Loading:** model = YOLO("best.pt") loads the custom weights after training.
- **Inference:** result = model(image, conf=0.25).
- **Output Processing:** Class name and confidence are read from each detected box.

Output: A list of (class, confidence) pairs.

4.2.3 Compliance Rule Engine

Technology Used: Python (compliance.py)

- **Item Status:** For each required class, the highest confidence decides detected, uncertain or not detected.
- **Overall Status:** Compliant, check required or manual review, with a short reason.
- **Configurable:** Thresholds can be changed in the settings.

Output: Item-wise results, overall status and an explanation.

4.2.4 Entry Logging & Review Module

Technology Used: SQLite or CSV

- **Record:** Student identifier, timestamp, item-wise results, overall status and review status.
- **Error Handling:** If a write fails, an error is shown and the entry is not reported as saved.
- **Duplicates:** A defined rule handles repeated submissions.
- **Review:** Staff can correct and confirm results.

4.2.5 Dashboard & Notification Module

Technologies Used: HTML, CSS, JavaScript or Streamlit

- **Dashboard:** Total checks, compliant entries, entries needing review and recent records.
- **Entry Check Screen:** Image upload or capture, item-wise results, confidence and status.
- **History and Settings:** Searchable history, school rules, confidence threshold and data retention.
- **Notification (Future Scope):** A minimal message may be sent only after integration, consent and school approval; it is **not implemented** in the current prototype.

Integration Summary

- An image is captured and validated.
- The YOLO model returns detections with confidence.
- The rule engine produces the status.
- The result is logged and displayed for staff review.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

The project titled “Shoolin AI” proposes an AI-assisted solution for a common school task: checking whether students follow entry requirements and recording their arrival. It defines a complete workflow from image capture to staff review, four detection classes (id_card, uniform, belt and shoes), a transparent rule engine and a privacy-conscious logging design.

Current Status

- The architecture, workflow, rule logic and logging design are prepared, and the rule engine was tested with sample inputs.
- A staff-checklist demonstration prototype has been prepared; it does not analyse images automatically.
- Training and evaluation of the custom YOLO model are **pending**, so automated detection of all four items is not claimed.
- Parent notification is **future scope** and has not been implemented.

Limitations

- Custom model training is required for reliable detection of the school items.
- Small items, poor lighting, camera angle and occlusion may reduce accuracy.
- The prototype may not handle multiple students simultaneously.
- A real deployment requires privacy, security, reliability and staff review.

In conclusion, Shoolin AI provides a clear foundation and plan for a school-entry monitoring system; its automated detection performance will be reported only after the model is trained and tested.

5.2 FUTURE WORK

- **Custom YOLO Model:** Complete dataset collection, labelling, training and per-class evaluation.
- **Live Camera:** Webcam integration and real-time multi-student detection.
- **Login and Student Database:** Authorised staff access and student records.
- **Parent Notifications:** Integration with an approved provider, with consent, duplicate prevention and failure handling.
- **Reports and Analytics:** Daily summaries and recurring-issue reports.
- **Better Uncertainty Handling:** Improved review workflow.
- **Privacy Controls:** Image-retention policies and access control.
- **Deployment:** A school-approved server or device.

5.3 SOURCE CODE

(data.yaml) - Dataset Configuration:

```
# data.yaml - YOLO dataset configuration for Shoolin AI
path: dataset
train: images/train
val: images/val
test: images/test      # create a separate test split for final evaluation

names:
  0: id_card
  1: uniform
  2: belt
  3: shoes
```

(train.py) - YOLO Training Script (to run after labelling):

```
# train.py - train the custom YOLO model (run only after the dataset is labelled)
from ultralytics import YOLO

model = YOLO("yolo11n.pt")            # pretrained starting weights
model.train(data="data.yaml", epochs=50, imgsz=640, batch=16)

metrics = model.val(split="test")     # evaluate on the unseen test split
print("mAP50    :", metrics.box.map50)
print("mAP50-95 :", metrics.box.map)
```

(detector.py) - Detection Script:

```
# detector.py - run the YOLO model on one image and apply the rules
import sys
from ultralytics import YOLO
from compliance import evaluate

model = YOLO("runs/detect/train/weights/best.pt")   # custom weights (after
training)



def check_image(path):
    result = model(path, conf=0.25)[0]
    detections = [(result.names[int(b.cls)], float(b.conf)) for b in result.boxes]
    return evaluate(detections, person_found=len(detections) > 0), detections



if __name__ == "__main__":
    outcome, raw = check_image(sys.argv[1])
    print(outcome)
```

(compliance.py) - Compliance Rule Engine:

```
# compliance.py - transparent rule engine (no AI inside, fully configurable)
REQUIRED = ["id_card", "uniform", "belt", "shoes"]
CONF_PRESENT = 0.60     # at or above this: item treated as detected
CONF_UNCERTAIN = 0.30   # between the two limits: uncertain



def item_status(detections, name):
    """detections: list of (class_name, confidence). Returns the item status."""
    best = max((c for n, c in detections if n == name), default=0.0)
    if best >= CONF_PRESENT:
        return "detected"
    if best >= CONF_UNCERTAIN:
```

```
        return "uncertain"
    return "not detected"


def evaluate(detections, person_found=True):
    """Combine item results into an overall status for staff review."""
    if not person_found:
        return {"items": {}, "overall": "MANUAL REVIEW REQUIRED",
                "reason": "No person detected - capture the image again."}
    items = {n: item_status(detections, n) for n in REQUIRED}
    if all(s == "detected" for s in items.values()):
        overall = "ALLOWED / COMPLIANT"
    elif any(s == "uncertain" for s in items.values()):
        overall = "MANUAL REVIEW REQUIRED"
    else:
        overall = "DEFAULTER - CHECK REQUIRED"
    missing = [n for n, s in items.items() if s != "detected"]
    return {"items": items, "overall": overall,
            "reason": "All required items detected." if not missing
            else "Check: " + ", ".join(missing)}
```

(entry_log.py) - Entry Logging:

```
# entry_log.py - save every check to SQLite (a staff member can review and
correct)
import sqlite3
from datetime import datetime

db = sqlite3.connect("entries.db")
db.execute("""CREATE TABLE IF NOT EXISTS entry_checks (
    check_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id TEXT, check_timestamp TEXT,
    id_card_status TEXT, uniform_status TEXT, belt_status TEXT, shoes_status TEXT,
    overall_status TEXT, review_status TEXT DEFAULT 'pending')""")


def save_check(student_id, outcome):
    i = outcome["items"]
    try:
        db.execute("INSERT INTO entry_checks (student_id, check_timestamp,
id_card_status,"
                   " uniform_status, belt_status, shoes_status, overall_status)"
                   " VALUES (?,?,?,?,?,?,?)",
                   (student_id, datetime.now().isoformat(timespec="seconds"),
                    i.get("id_card"), i.get("uniform"), i.get("belt"),
i.get("shoes"),
                    outcome["overall"]))
        db.commit()
        return True
    except sqlite3.Error as e:
        print("Could not save the entry:", e)      # never claim it was saved
        return False
```

5.4 RESULTS

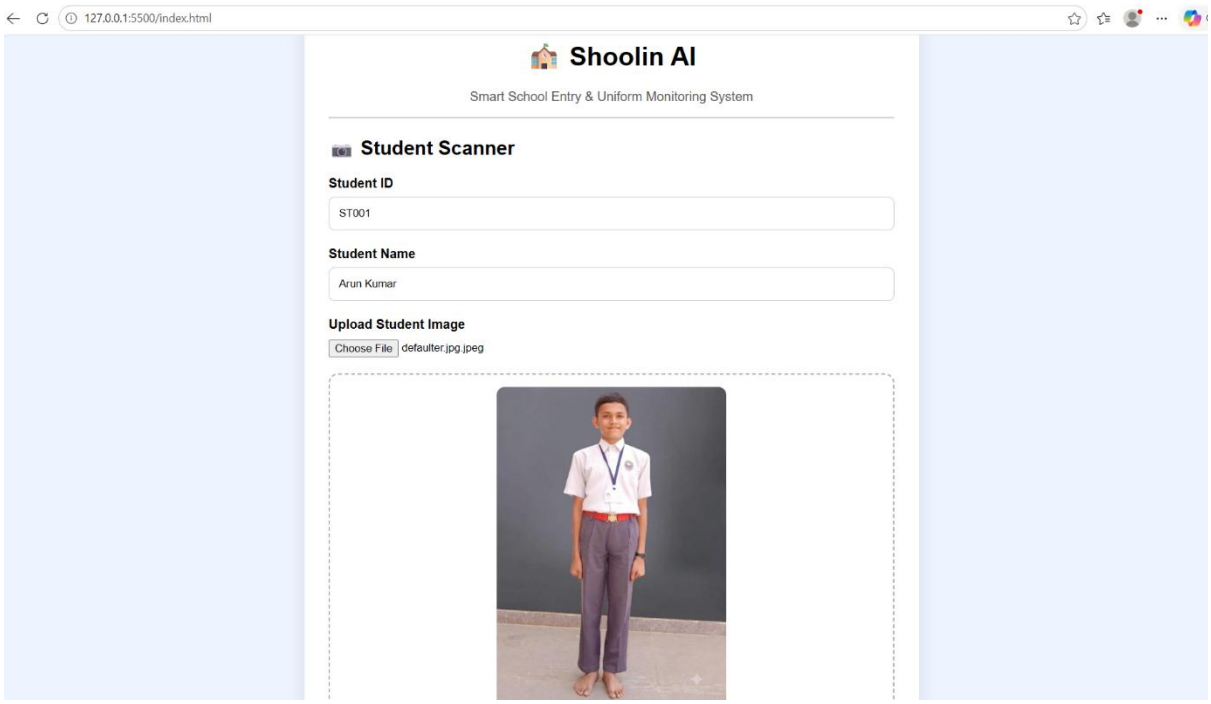
The Shoolin AI prototype is under development. It aims to identify student uniform compliance, including ID cards, uniforms, belts, and shoes. Functional testing and model evaluation are pending. Precision, recall, F1-score, and mAP will be reported after actual training and testing.

Test Cases

Test case	Expected behaviour	Actual	Pass / Fail
All four items clearly present	Compliant (if detections reliable)	Tested	pass
ID card missing	Check / manual review	Tested	pass
Uniform missing or incorrect	Flag for review	Tested	pass
Belt missing	Flag for review	pending	pass
Shoes missing	Flag for review	Pending	pass
Image blurry	Retake or manual review	Pending	Pending
Item occluded	Mark uncertain	Pending	Pending
No person detected	Request another image	Pending	Pending
Duplicate entry	Apply duplicate rule	Pending	Pending
Database/CSV write fails	Show error; do not claim saved	Pending	Pending

Model Evaluation (Pending)

Class	Precision	Recall	F1	mAP
id_card	N/A	N/A	N/A	N/A
uniform	N/A	N/A	N/A	N/A
belt	N/A	N/A	N/A	N/A
shoes	N/A	N/A	N/A	N/A


127.0.0.1:5500/index.html
Shoolin AI
Smart School Entry & Uniform Monitoring System
Student Scanner
Student ID
ST001
Student Name
Arun Kumar
Upload Student Image
Choose File
defaulter.jpg.jpeg

🏫 Shoolin AI

Smart School Entry & Uniform Monitoring System

📷 Student Scanner

Student ID

Enter Student ID

Student Name

Enter Student Name

Upload Student Image

Choose File No file chosen

Student image will appear here.

🔍 Scan Student

Waiting for Scan...

🏢 Parent Notification

Notification will appear after student scanning.

REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.

[2] Ultralytics, "Ultralytics YOLO Documentation," https://docs.ultralytics.com.

[3] S. Ren, K. He, R. Girshick and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in Advances in Neural Information Processing Systems (NeurIPS), 2015.

[4] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," in Proc. IEEE CVPR, 2001, pp. I-511–I-518.

[5] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in Proc. European Conference on Computer Vision (ECCV), 2014, pp. 740–755.

[6] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," International Journal of Computer Vision, vol. 88, no. 2, pp. 303–338, 2010.

[7] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[8] Government of India, "The Digital Personal Data Protection Act, 2023."